

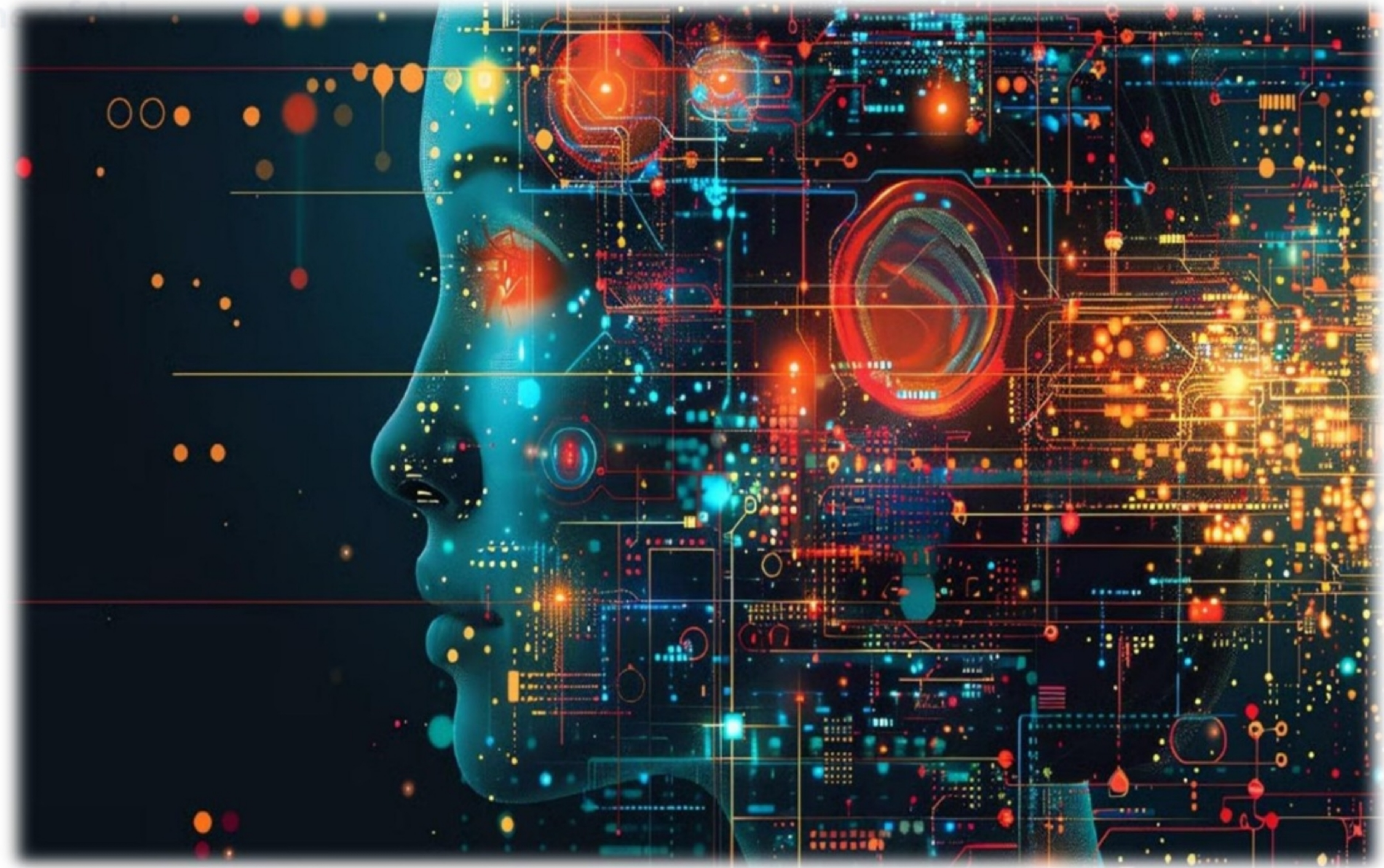
Linear Algebra – The Backbone of AI/ML

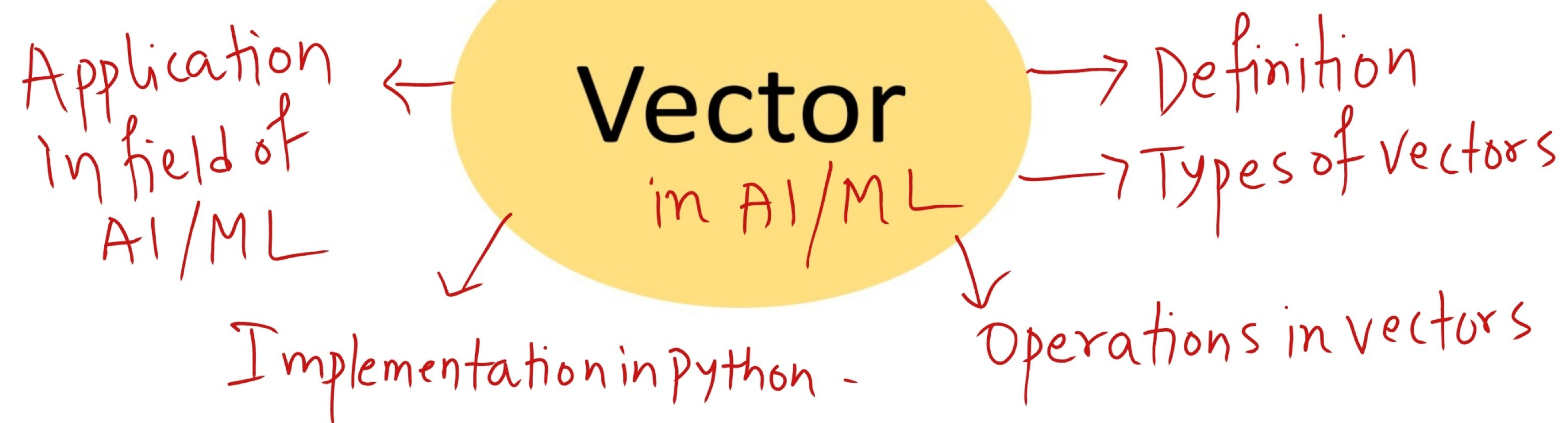
Linear Algebra – The Backbone

TOPIC-

Vector, properties and operation

Sanjay Kumar Sen
Assistant Professor at
Quad AI





Problem:-

$$P = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix}$$

Then P^{50} is = ?

$$P^2 = \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$$

$$P^3 = P^2 \cdot P = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ \frac{3}{2} & 1 \end{bmatrix}$$

$$P^4 = P^3 \cdot P = \begin{bmatrix} 1 & 0 \\ \frac{3}{2} & 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 \\ \frac{1}{2} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix}$$

$$P^{50} = \begin{bmatrix} 1 & 0 \\ \frac{50}{2} & 1 \end{bmatrix}$$

$$P^{50} = \begin{bmatrix} 1 & 0 \\ 25 & 1 \end{bmatrix} \text{ Ans}$$

$$\text{let, } A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

Then, $A^{2025} - A^{2020}$ is equal to = ?

~~(A)~~ $A^6 - A$
~~(B)~~ A^5
~~(C)~~ $A^5 - A$
~~(D)~~ A^6

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$A^2 = A \cdot A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$A^3 = A^2 \cdot A = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 2 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$A^n = \begin{bmatrix} 1 & 0 & 0 \\ n-1 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix} \quad A^{2025} = \begin{bmatrix} 1 & 0 & 0 \\ 2024 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}, \quad A^{2020} = \begin{bmatrix} 1 & 0 & 0 \\ 2019 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

$$A^{2025} - A^{2020} = \begin{bmatrix} 0 & 0 & 0 \\ 5 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

$(1 \times 1 + 0 \times 0 + 0 \times 1)$

Matrix Multiplication

$$A^6 = \begin{bmatrix} 1 & 0 & 0 \\ s & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

(A curved arrow from the top-left element 1 to the top-right element 0 is labeled $n-1$)

$$A' = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

(A curved arrow from the top-left element 1 to the top-right element 0 is labeled $n-1$)

$$A^6 - A = \begin{bmatrix} 0 & 0 & 0 \\ s & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \text{ Ans}$$

$$S = \begin{bmatrix} 1 & 0 & 0 \\ 4 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$$

(A curved arrow from the top-left element 1 to the top-right element 0 is labeled $n-1$)

$A^6 = X$

What is a Vector?

A vector is a tuple of one or more values called scalars. ✓

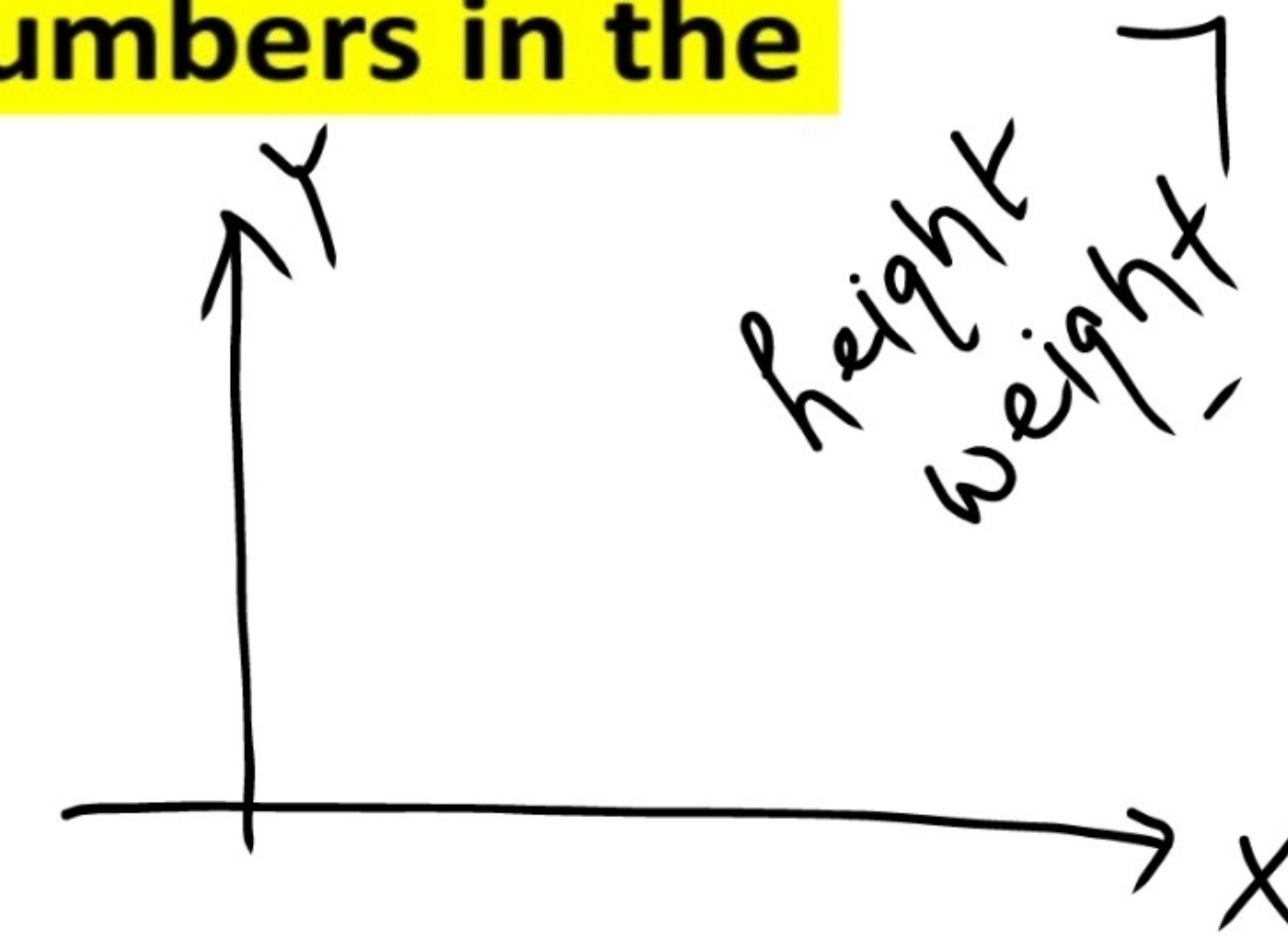
“Vectors are built from components, which are ordinary numbers. You can think of a vector as a list of numbers, and vector algebra as operations performed on the numbers in the list”

25

A vector in a two-dimensional plane can be defined as:

$$v = (x_1, x_2)$$

$$v = (170, 65)$$



In higher dimensions, vectors will have more components, such as: ✓

$$v = (x_1, x_2, x_3, \dots, x_n)$$

where n is the number of features of a specific data point.

$$v = (x_1, x_2, x_3, \dots, x_n)$$



Code to Create and Print a Vector in NumPy

1 Scaler

scalar = 1



1
2 Vector

1-D Array:- `np.array([1,2])`



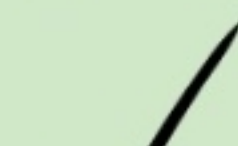
1 1
2 2 Matrix

`np.array([[1,1],[2,2]])`



1 1 3
2 2 4 Tensor

`np.array([[[1,1],[2,2]],
[[3,3],[4,4]])`



2D $\rightarrow \begin{bmatrix} 1 & 1 \\ 2 & 2 \end{bmatrix}$

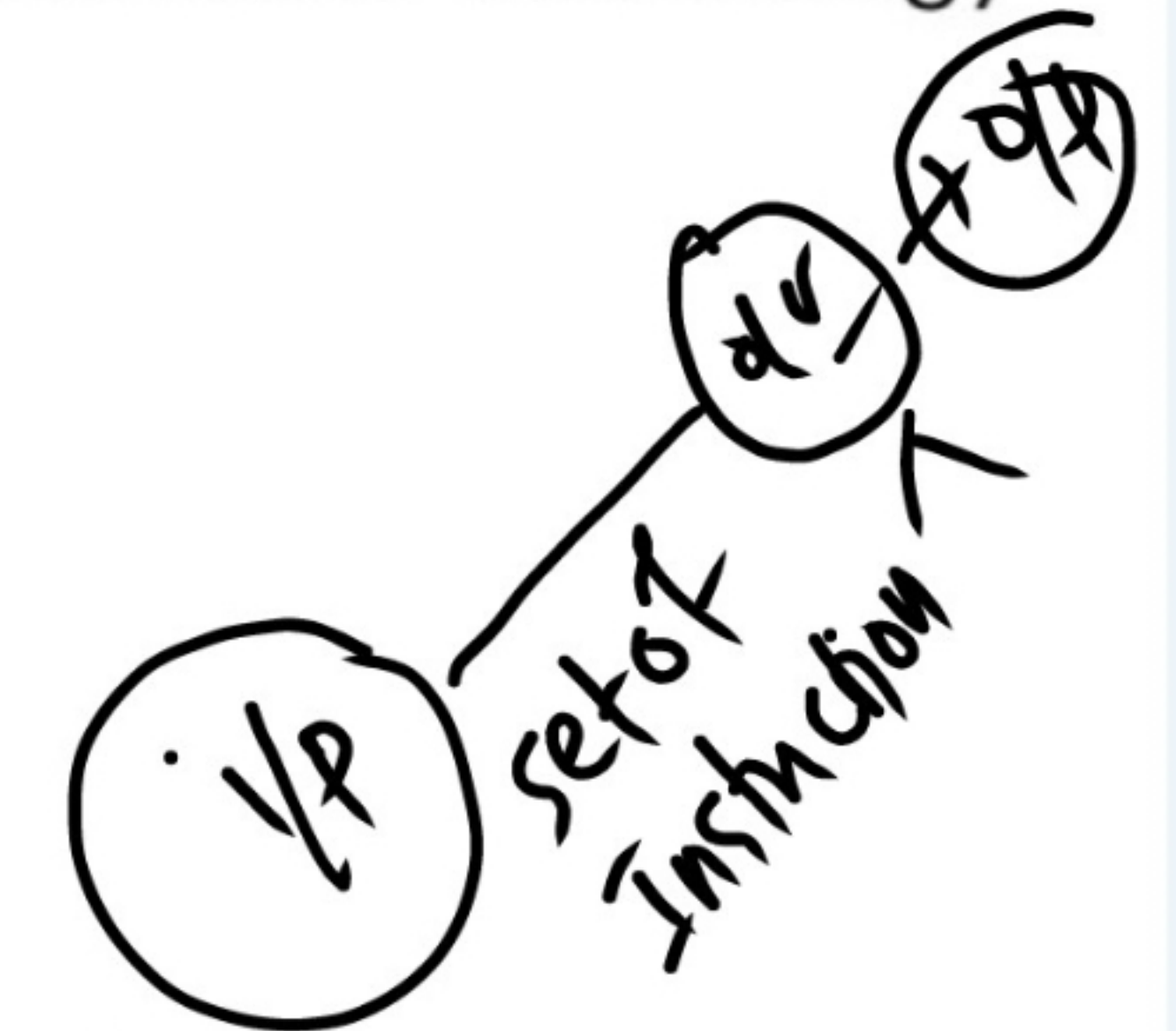
By using python :-
(create a vector:-)

import numpy as np

vector = np.array([170, 65])

print("Vector:", vector)

$\frac{I}{P} \rightarrow \text{height} \rightarrow \text{weight}$
 $\downarrow \qquad \qquad \downarrow$
 170 $\qquad \qquad$ 65
 vector: [170, 65]



Scalars, Vectors and Matrices

Scalars: Any single value from our dataset would represent a scalar like integers or floating-point numbers, employed in mathematical computation. ✓ ✓

Vectors: Vectors are one-dimensional number arrays that hold several values in a linear sequence. ✓

Matrices: Matrices are two-dimensional arrays of multiple vectors that are placed in rows and columns. ✓

$$\text{Scalar} = [25]$$

$$\text{Matrix} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix}_{2 \times 2}$$

$$V = [170 \ 65]_{1 \times 1}$$

↓
vector



These mathematical structures play an essential role in machine learning models, facilitating effective calculations and data representation.

Create a Matrix by using python :

import numpy as np

Define a 3x3 matrix

**mat = np.array([[1, 2, 3],
[4, 5, 6],
[7, 8, 9]])**

i/p

print(mat)

O/p:-

O/p:-

Matrix:

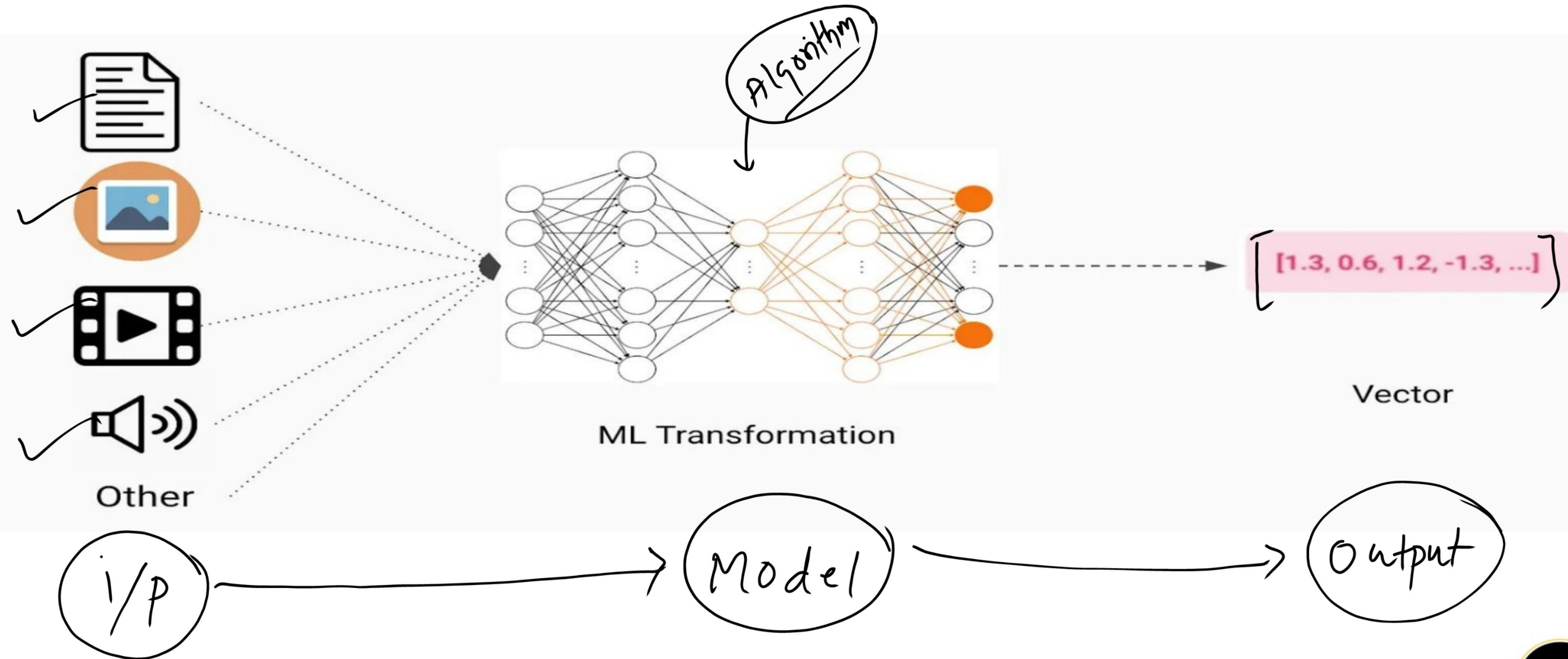
$\begin{bmatrix} [1 \ 2 \ 3] \\ [4 \ 5 \ 6] \\ [7 \ 8 \ 9] \end{bmatrix}$

matrix = $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}_{3 \times 3}$

Google Colab
Full build
numpy
Pandas
Matplotlib
Seaborn



Vectors in Machine Learning Models



Types of Vectors in Machine Learning

1. Row and Column Vectors

A row vector is a one-dimensional array represented in a row format: (x_1, x_2, x_3)

* column: $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$

2. Zero Vector

$v = (0, 0, 0)$, All elements are zero



3. Unit Vector

A vector of magnitude 1. It is frequently used to denote direction:

$$u = v / \|v\|$$

$$u = \frac{v}{\|v\|}$$

where $\|v\|$ is the magnitude of vector v .

$$\text{magnitude} = \sqrt{29}$$

$$\begin{aligned} v &= 2\hat{i} + 3\hat{j} + 4\hat{k} \\ \|v\| &= \sqrt{2^2 + 3^2 + 4^2} \\ &= \sqrt{4 + 9 + 16} \\ &= \sqrt{29} \end{aligned}$$



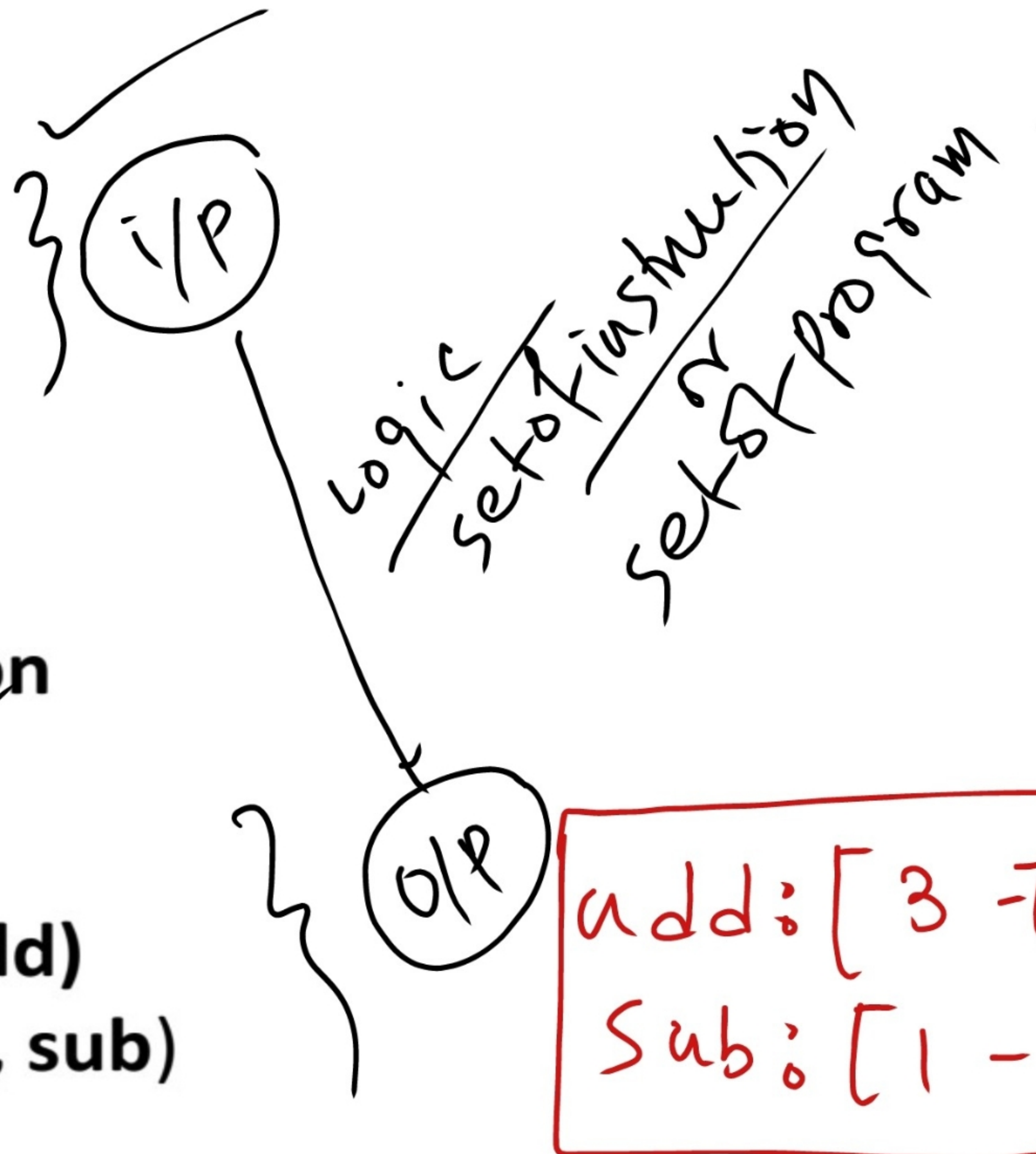
Vector Operations in Machine Learning

vector Addition and subtraction using python

```
import numpy as np
a = np.array([2, 3])
b = np.array([1, 4])
```

```
# Perform addition
add = a + b
# Perform subtraction
sub = a - b
```

```
print("Addition:", add)
print("Subtraction:", sub)
```



$$\text{vec}_a = [2 \ 3]$$

$$\text{vec}_b = [1 \ 4]$$

$$\text{add} = [2+1 \ 3+4]$$

$$= [3 \ 7] \checkmark$$

$$\text{Substraction} = [2-1 \ 3-4]$$

$$= [1 \ -1]$$



Scalar Multiplication :-

```
import numpy as np
```

```
a = np.array([1, 2, 3])  
scalar = 3
```

```
# Perform scalar multiplication
```

```
res = scalar * a
```

```
print("Scalar Multiplication:", res)
```

O/p res: $\begin{bmatrix} 3 & 6 & 9 \end{bmatrix}$

Constant = Scalar
↓
Multiplication

$$V = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

$$3 \cdot V = \begin{bmatrix} 3 \times 1 & 3 \times 2 & 3 \times 3 \end{bmatrix} \\ = \begin{bmatrix} 3 & 6 & 9 \end{bmatrix}$$



10/9/25

DPP-03 ∴

① $\vec{a} = 3\hat{i} + 4\hat{j}$

$$\begin{aligned}\text{Magnitude} &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} \\ &= 5 \quad \underline{\text{Ans}}\end{aligned}$$

② $\vec{v} = (1, 2, 3),$

$$\begin{aligned}\text{length}(\vec{v}) &= \sqrt{1^2 + 2^2 + 3^2} \\ &= \sqrt{1 + 4 + 9} \\ &= \sqrt{14} \quad \checkmark\end{aligned}$$

③

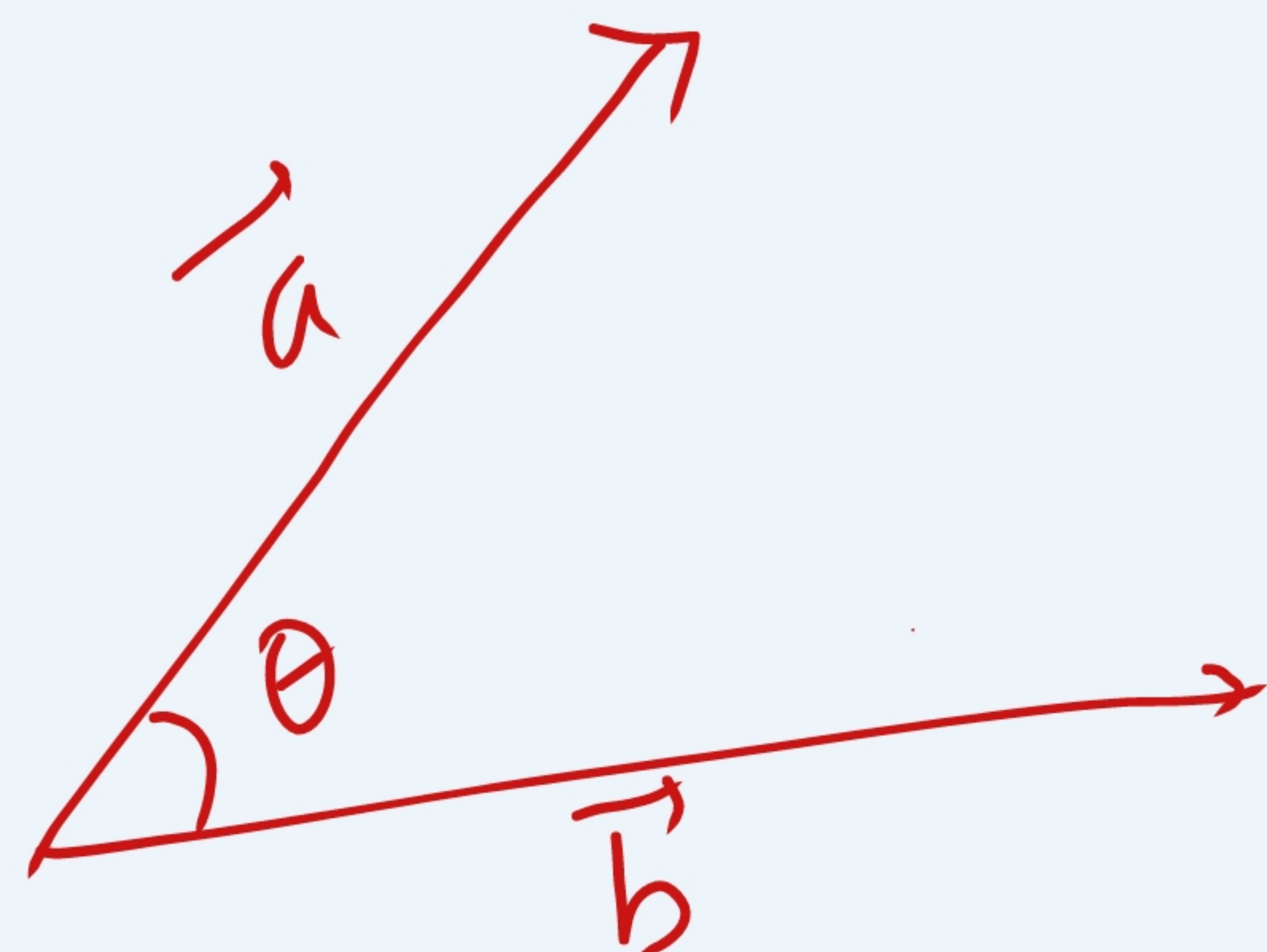
$$\vec{a} + \vec{b} = (5+2)\hat{i} + (3+1)\hat{j} + (4+2)\hat{k} = (7\hat{i} + 4\hat{j} + 6\hat{k}) \checkmark$$

$$\vec{a} - \vec{b} = (5-2)\hat{i} + (3-1)\hat{j} + (4-2)\hat{k} = (3\hat{i} + 2\hat{j} + 2\hat{k}) \checkmark$$

like an example:-

$$\begin{aligned} \vec{a} &= 5\hat{i} + 3\hat{j} + 4\hat{k} \\ \vec{b} &= 2\hat{i} + 1\hat{j} + 2\hat{k} \end{aligned}$$

Scalar product :-



$$ab = |a||b|\cos\theta$$

a = magnitude of vector a

b = " " of " " b

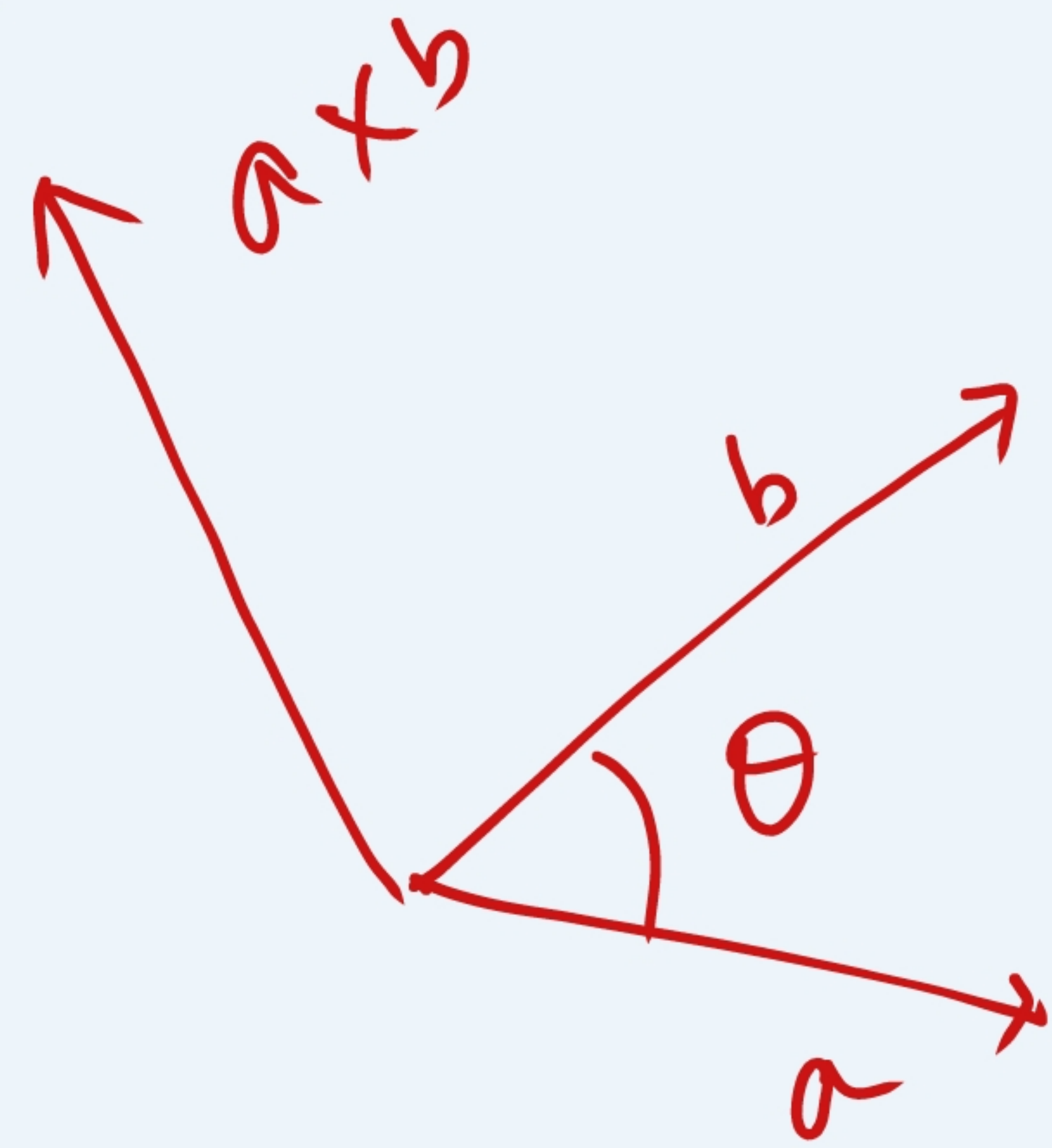
θ = angle between the vector

Ex 1: $|a| = 6$
 $|b| = 3$
 $\theta = 60^\circ$
Then their dot product?
 $ab = 6 \cdot 3 \cdot \cos(60^\circ)$
 $ab = 18 + 2$
 $ab = 9$ ✓

Ex 2: $a = 3\hat{i} + \hat{j} - 4\hat{k}$
 $b = 8\hat{i} - 8\hat{j} + 4\hat{k}$ are perpendicular ✓ ✓

$$3 \times 8 + 1 \times (-8) + (-4) \cdot (4)$$
$$= ?$$

Cross product



$$a \times b = |a| |b| \sin \theta$$

$$a \times b = (a_2 b_3 - a_3 b_2, a_3 b_1 - a_1 b_3, a_1 b_2 - a_2 b_1)$$

$$a \times b = \begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{vmatrix}$$

$$= \hat{i} (\quad) - \hat{j} (\quad) + \hat{k} (\quad)$$

Find the area of parallelogram whose adjacent sides are

$$\begin{cases} \vec{a} = 4\hat{i} + 2\hat{j} - 3\hat{k} \\ \vec{b} = 2\hat{i} + \hat{j} - 4\hat{k} \end{cases}$$

$$\begin{vmatrix} + & - & + \\ \hat{i} & \hat{j} & \hat{k} \\ 4 & 2 & -3 \\ 2 & 1 & -4 \end{vmatrix}$$

$$\textcircled{(-8+3)}\hat{i} - \hat{j}(-16+6) + \hat{k}(4-4)$$

$$\begin{aligned} & -5\hat{i} - (-10)\hat{j} + 0 \\ & 5\hat{i} + 10\hat{j} = \sqrt{5^2 + 10^2} = \sqrt{125} = 5\sqrt{5} \\ & \quad \quad \quad = \sqrt{25+100} = \sqrt{125} \Rightarrow 5\sqrt{5} \end{aligned}$$

Dot product using python :-

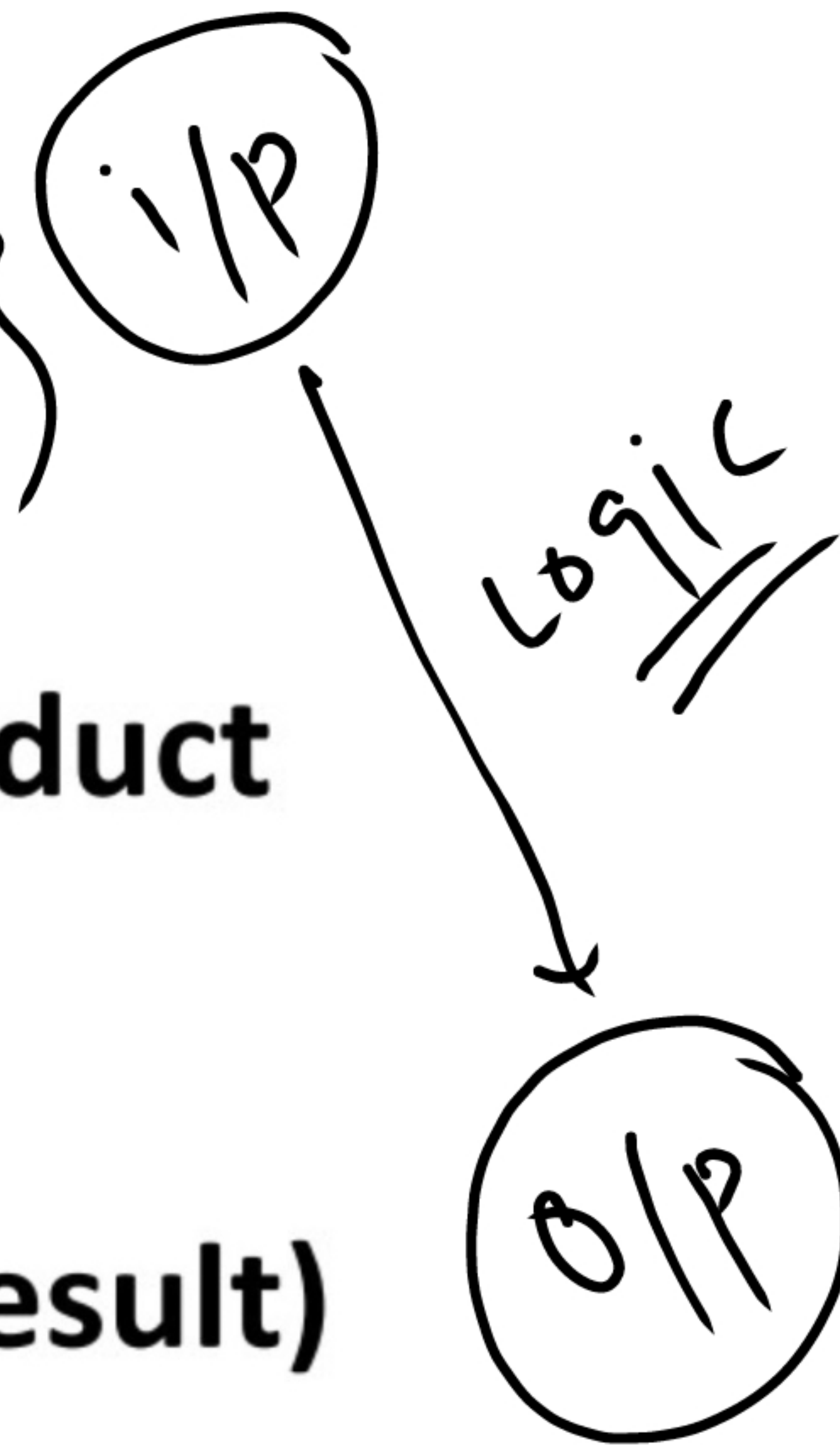
```
import numpy as np
```

```
a = np.array([1, 2, 3])  
b = np.array([4, 5, 6])
```

```
# Compute the dot product  
result = np.dot(a, b)
```

```
print("Dot Product:", result)
```

O/p:- result: 32



$$a = [1, 2, 3]$$

$$b = [4, 5, 6]$$

$$\begin{aligned}
 a \cdot b &= [1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6] \\
 &= 4 + 10 + 18 \\
 &= 32 \text{ Ans}
 \end{aligned}$$



Cross product using python

Code

```
import numpy as np
```

```
c = np.array([1, 2, 3])
```

```
d = np.array([4, 5, 6])
```

```
# Compute the cross product
```

```
res = np.cross(c, d)
```

```
print("Cross Product:", res)
```

result = $[-3 \ 6 \ -3]$

Ex:

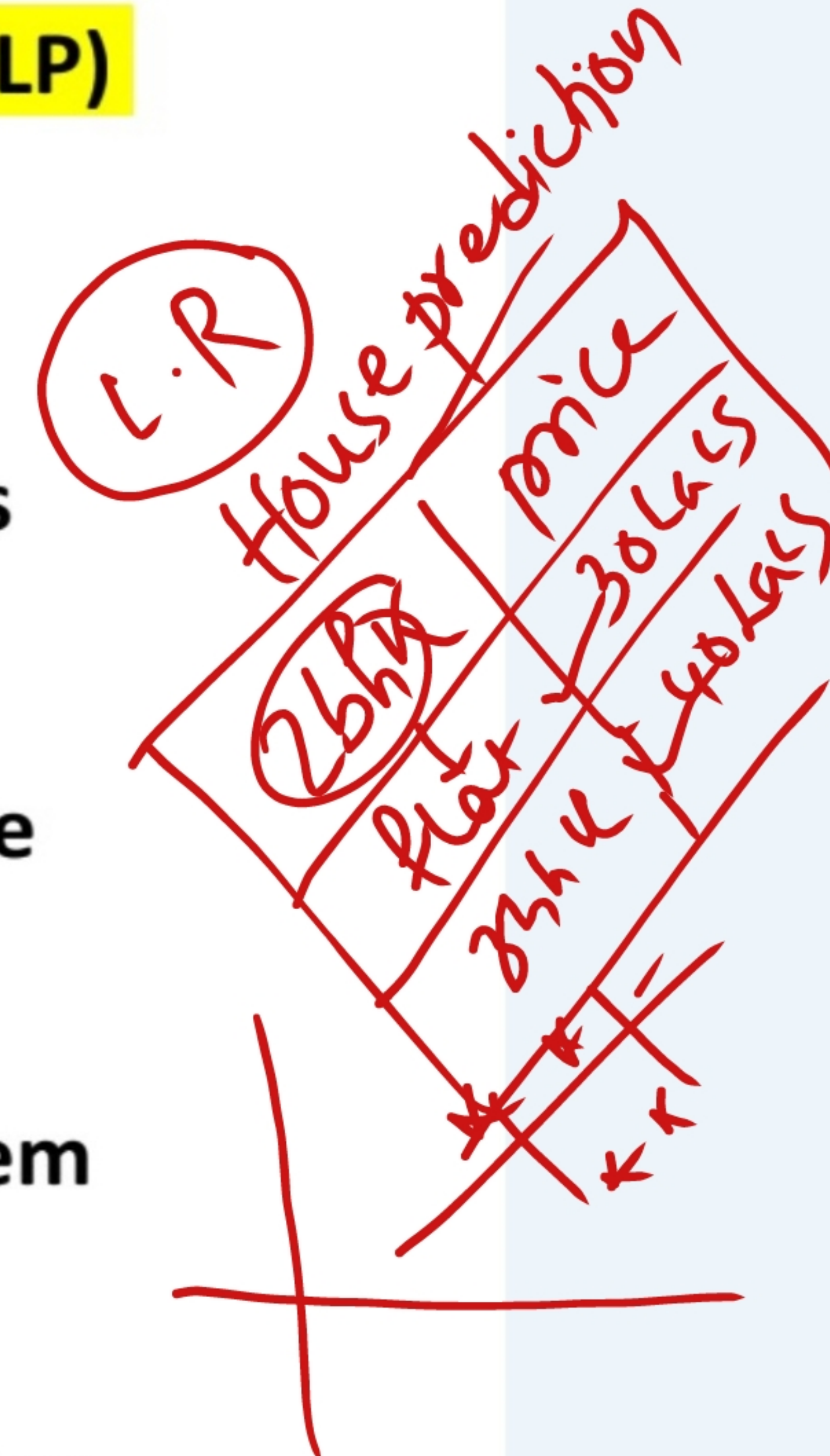
$$\begin{vmatrix} \hat{i} & \hat{j} & \hat{k} \\ 1 & 2 & 3 \\ 4 & 5 & 6 \end{vmatrix}$$

$$\begin{aligned} &= \hat{i}(12-15) - \hat{j}(6-12) + \hat{k}(5-8) \\ &= \hat{i}(-3) - \hat{j}(-6) + \hat{k}(-3) \\ &= -3\hat{i} + 6\hat{j} - 3\hat{k} \\ &= [-3 \ 6 \ -3] \end{aligned}$$



Vectors play a crucial role in various machine learning algorithms and natural language processing (NLP) techniques.

- **Linear Regression**- Linear regression employs vectors to denote the independent and dependent variable relationship: $Y = Xw + b$ where X is a feature vector, w is a weights vector and b is the bias term.
- **Support Vector Machines (SVMs)**- SVMs utilize vector mathematics to identify the best hyperplane separating various classes in classification problems.
- **Neural Networks**- Neural networks store weights, activations and gradients as vectors, making them crucial for deep learning models.
- **Clustering (K-Means Algorithm)**- The K-Means algorithm allocates points to clusters based on vector distances.
- **Word Embeddings in NLP**- Methods such as Word2Vec and GloVe map words to vectors, capturing their semantic meaning.



SUMMARY

1) Addition

$$\vec{a} + \vec{b} = (a_x + b_x)\hat{i} + (a_y + b_y)\hat{j} + (a_z + b_z)\hat{k}$$

2) Subtraction

$$\vec{a} - \vec{b} = (a_x - b_x)\hat{i} + (a_y - b_y)\hat{j} + (a_z - b_z)\hat{k}$$

3) Scalar Multiplication,

$$K\vec{a} = (Ka_x)\hat{i} + (Ka_y)\hat{j} + (Ka_z)\hat{k}$$

4) dot product, $\vec{a} \cdot \vec{b} = |\vec{a}| |\vec{b}| \cos \theta = a_x b_x + a_y b_y + a_z b_z$

5) Cross product,

$$\vec{a} \times \vec{b} = |\vec{a}| |\vec{b}| \sin \theta$$

1) Magnitude -

$$|\vec{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

$$\begin{aligned} &= \sqrt{3^2 + 4^2} \\ &= \sqrt{9 + 16} \\ &= \sqrt{25} \\ &= 5 \text{ Ans} \end{aligned}$$